Understanding AI Agents

The WHAT and HOW

Course Goals

Understand AI Agents

What they actually are

A definition that holds up — and what the word gets used for that isn't one.

How they work inside

We open one up and follow a single run, one lap at a time.

The kinds you'll meet

Broad everyday helpers, narrow one-job agents, and where each fits.

Working with them

How you steer them, what they need from you, and where they need watching.

Agents vs workflows

Where the line runs, and why picking the wrong side costs you.

How to build them

The pieces they are made of, and how those fit together.

No deep technical dive — just the **core ideas**, also from behind the scenes.

What is an AI Agent?

An AI agent is a tool that takes on a whole task **by itself** — using AI to work out every next step.

What Is An AI Agent?

A piece of software

A desktop application, a CLI tool — something you can run.

+

Uses an LLM

The "brain" of the agent, making the decisions and controlling the action.

+

Information & tools

The knowledge and tools the model needs to complete its task.

TWO KINDS OF THEM

General

One agent, any job. Broad tools, and you point it at whatever you need today.

Task-specific

One job, done the same way every time. Narrow tools, instructions written once.

TWO PLACES THEY RUN

On your machine

Started by you, working on your files and your accounts, while you watch.

On a remote server

Set off by a message, an email or a schedule, with nobody watching.

Agent, Harness & Agent Loop

Those apps contain the **harness**. What they run inside is the **agent loop**.

AGENT HARNESS — THE SOFTWARE THAT FEEDS THE LOOP

takes your prompt

adds the instructions

describes the tools

CONTEXT

Agent Loop

ask the LLM
↓ it names a tool
↓ feed the result back

Round and round until **it is done.**

TOOL CALL

RESULT

runs the tool it asks for

hands the result back

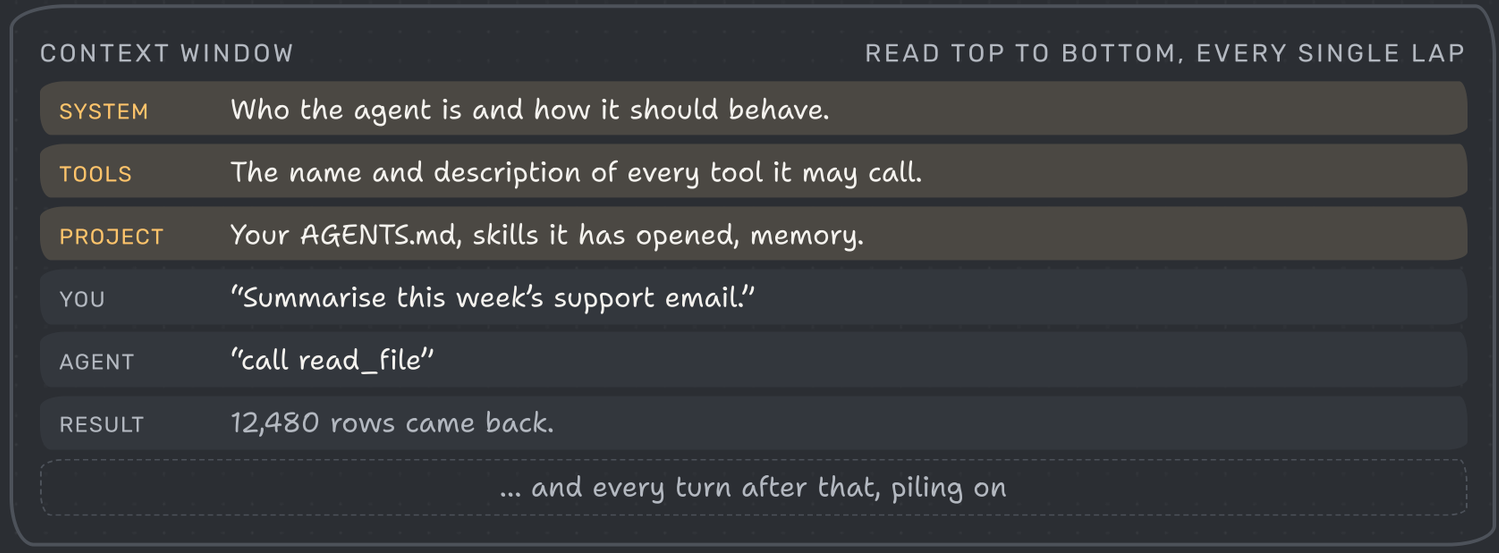
keeps the whole history

Understanding Agent Context

You work with an agent in **sessions** — and every session has one **context window** that everything is written into.

ONE SESSION

One conversation, one window. Start a new one and it is empty again.



The top three are there **before you type a word** — put there by the harness.

Running Agents in Different Environments

That harness is just software — so it can run **wherever you put it.**

On your machine

IN	You type it, and you watch it work
RUNS	**Your laptop** often inside a sandbox
REACHES	Your files, your terminal, your logged-in accounts

Usually the broad, everyday kind — it sits where you work

On a server

IN	A Slack message, an incoming email, a schedule
RUNS	**A remote server** nobody watching
REACHES	Company systems, a database, an API — not your laptop

Usually the narrow, one-job kind — it sits where the work arrives

Same loop either way. What changes is **who starts it, and what it can reach.**

Two Types Of Agents

Same loop inside — what changes is **how much freedom** it gets.

General agents

One agent, any job. Broad tools, and you steer it with a prompt.

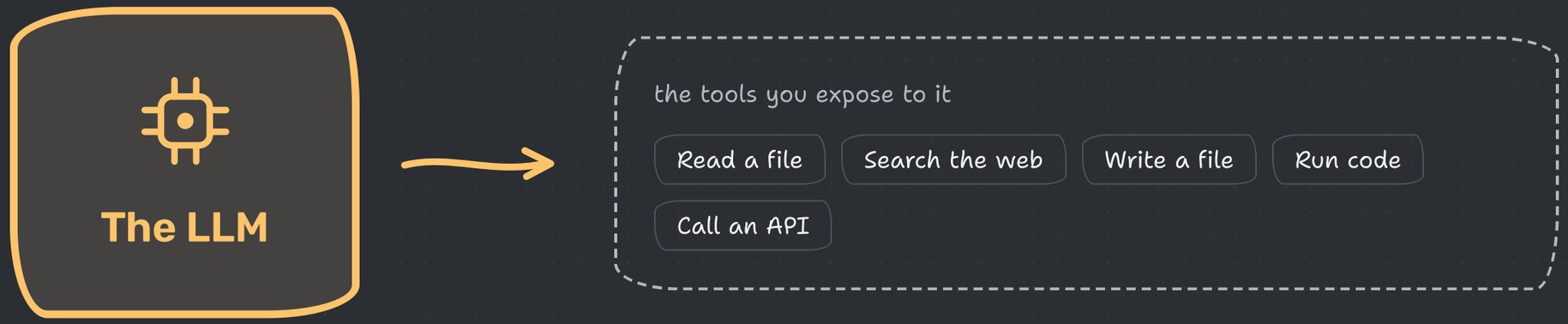
Task-specific agents

One job, done the same way every time. Narrow tools, instructions written once.

- Customer support
- Invoice processing
- Research

An Agent = AI + Its Tools

Inside every agent: an LLM, plus a set of tools **handed to it**.



Those tools decide **which tasks it can tackle** — nothing else does.

What The Tools Unlock

No tools

—

It can write text

An answer, a draft, an explanation. That's the whole list.

Give it

Read a file | Search the web

Now it can research

It digs through your documents and goes out to the web.

Give it

Write a file | Run code

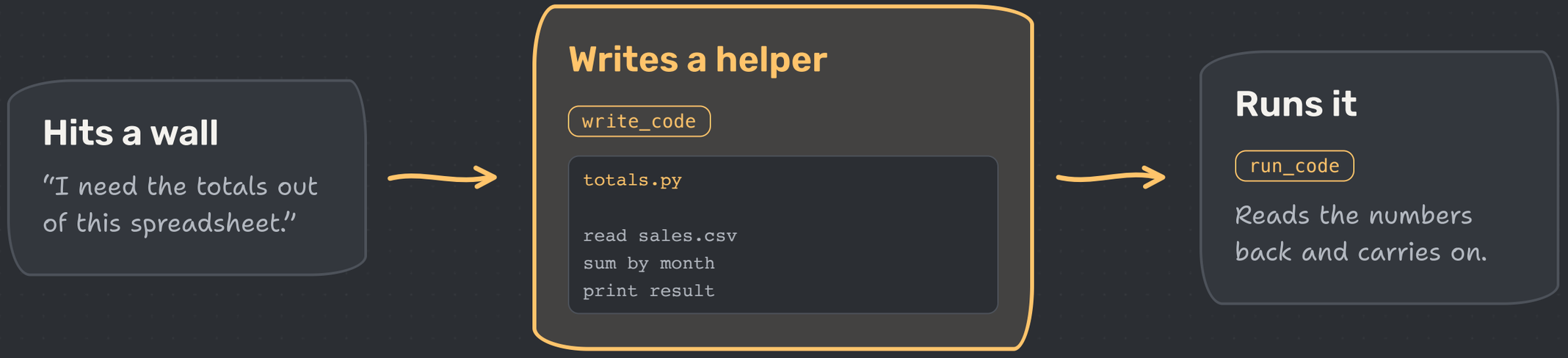
Now it can build

It writes its own little programs and runs them.

fewer tools — more it can take on

It Writes Its Own Tools

Missing a tool for the job? It just **writes one** — with the write-code and run-code tools you handed it.



Most agents do this constantly — you usually never see it happen.

A Typical Agent Run

"Give me some summary statistics for **sales.csv**"

That is everything it gets — one file name, no instructions on how.

1. `read_file` — Opens **sales.csv** and sees what is actually in it.
2. `write_code` — Writes a short program to crunch exactly those columns.
3. `run_code` — Runs it, reads the output — and hands you the statistics.

Two kinds of tool. That is a whole AI agent at work.

But not every tool that uses AI is an agent.

AI Agents

vs

AI-powered Workflows

What's An AI Agent

An AI agent turns one prompt into an **agentic loop**: an LLM keeps choosing a tool, reading the result and deciding what to do next — until your goal is reached.

The LLM can only reach for tools **you explicitly hand it**:

Web search | File search | Read & write files | Run code | Call an API

Any Input Kicks It Off

A prompt you type is just the most obvious trigger.

A prompt
"Analyze this CSV for me."

A support email
"My order never arrived..."

Anything else
A file drops in, a timer fires, another program calls in.

The Agentic Loop

Starts up — and from here on it runs the same way, whatever came in.

What The Model Actually Gets

Every lap of the loop is **one call to the model** — and this is what rides along with it.

one request to the model

1. **The input**
   Your prompt — or that support email.
2. **The tools it may use**
   `read_file` `web_search` `run_code`
   Just their names and descriptions — never the code behind them.
3. **Whatever else you hand it**
   Instructions, memory, earlier steps — more on those later.

→

It Decides

The model reads all of it and answers one question: **what should happen next?**

The Harness Runs The Agent

An agent is really **two separate pieces** of software talking to each other.

THE PROGRAM ON YOUR SIDE

The Harness

It owns the loop: it builds the request, reads the answer, and runs whatever the model asks for.

your own code

SOMEWHERE ON A SERVER

The Model

Text in, text out. It never touches your files, your inbox or the web.

reached over an API

One lap = one request out, one response back.

The Model Never Runs A Tool

This is the part **almost everybody gets wrong**.

✕ **NOT THIS**

The Model → runs the search itself

It has no hands. It cannot open a file, run code, or reach the internet.

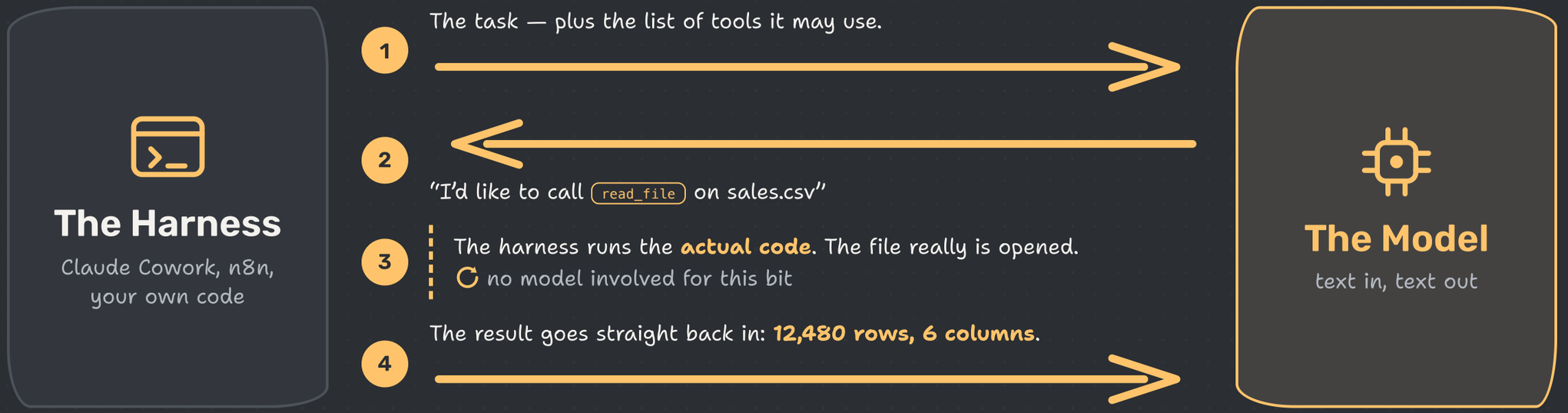
✓ **THIS**

The Model → asks for `web_search(…)` → The Harness runs it

All the model ever produces is text. A tool call is just text the harness knows how to act on.

The model **decides.** The program **does.**

One Lap Of The Loop



That is one lap. Now the harness asks again — result included.

Every Lap It Knows More

Over time, the model gains more and more **knowledge.**

Lap 1

- the prompt
- the tool list

→ "call read_file"

Lap 2

- the prompt
- the tool list
- it called read_file
- 12,480 rows came back

→ "call run_code"

Lap 3

- ... all of the above
- it ran the code
- totals per month

→ "I have everything I need"

Same model, same loop — only the pile it reads keeps growing.

Two Ways A Lap Can End

Every response is text. The harness checks **one thing**: was a tool asked for?

The Model Answers

Back to the harness, which reads it and looks for a tool call.

a tool was asked for

Round again

The harness runs it, adds the result, and calls the model one more time.

nothing was asked for

Done

Nothing left to run, so the harness stops looping and hands the answer over.

Nobody sets the number of laps up front — **the model does, one lap at a time.**

What You Get At The End

The last thing out of the loop is **always a message**. It is rarely the only thing you got.

Always: a message

The model's final answer, in text. Every agent run ends this way.

> "Done — revenue peaked in March, and I saved the breakdown next to your file."

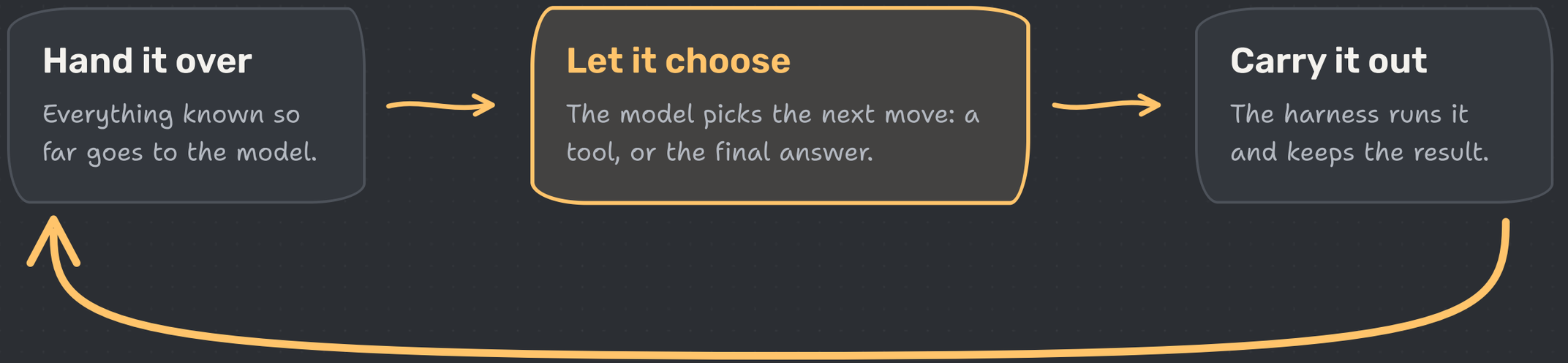
Usually also: real changes

Whatever the tools did along the way is already there — and stays there after the loop stops.

- a file on disk
- code that runs
- a row in a database
- an email that went out

The message is the **report.** The work happened during the laps.

That’s The Whole Trick



Repeat until the task is done. That is an AI agent.

Why This Is Worth Knowing

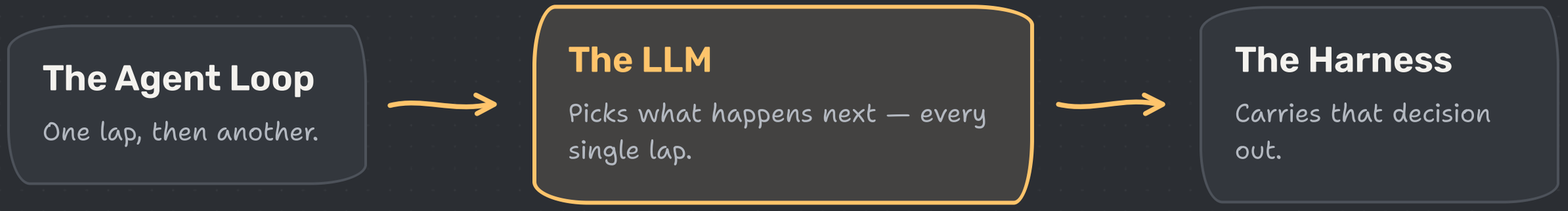
See the loop, and an agent stops being a black box — you can **read what it is doing.**

1. `web_segment` — The model **asked** for it — the harness is out running it right now.
2. `write_file` — That is a **real file** being written, not a description of one.
3. plain text — No tool asked for — the loop is **over**, and this is your answer.

The same loop is running inside **every one of them.**

The Loop Runs On An LLM

An agent is the loop — and a model **making every decision** inside it.

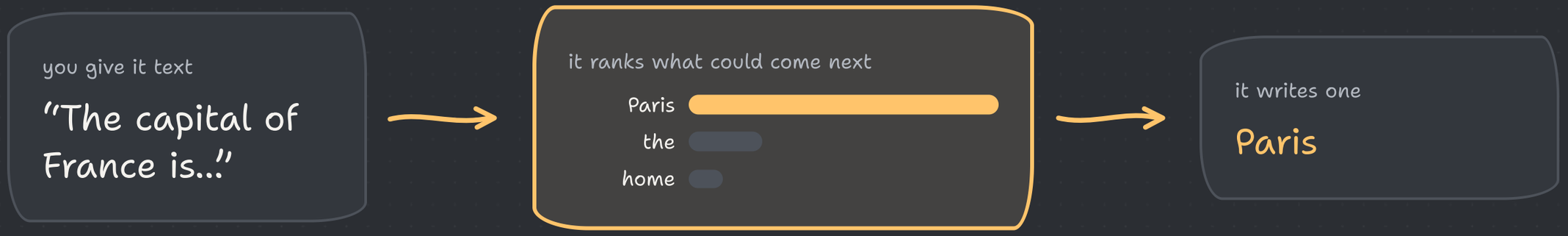


So everything hinges on that model — **what is an LLM, exactly?**

Quick Refresher: What's an LLM?

Strip away the magic and it's a **next-word guesser.**

Words — or **tokens**, to be precise — guessed from the context it has.



That is already how the very first ChatGPT worked, back in **2022**.

Then it starts over with "Paris" added. Word by word — that's the whole trick.

Probability, Not Lookup

Nothing is looked up. Every next word is a **probability.**

Those odds were **baked in during training** — from an enormous pile of text:

the web | books | code | ... and much more

All It Can Produce Is Text

So how does a next-word guesser end up **calling tools?**

Never anything else:

~~open a file~~ ~~run code~~ ~~send an email~~ ~~reach the web~~

True in 2022. **Still true today.**

Text Is Enough To Think In

Writing text is enough to **replicate a thought process.**

Thinking, written out

The model writes its reasoning the same way it writes anything else — one word at a time.

> "The user wants X… so the sensible next move is…"

And its tools ride along

The list of tools it may use is sent in **exactly like the prompt is** — just more text.

`read_file` `run_code` `web_search`

So it can reason about tools it **knows it has.**

How It Picks A Tool

"Analyze the data in **sales.csv**"

Plus the tool list — that is everything it has.

its simulated thinking, in plain text

1. The request is about data sitting in a file.
2. One tool is described as *opens a file and returns what is in it.*
3. So: `read_file` makes sense here.

A starting point for that CSV — reasoned, not hard-coded.

It Asks. The Program Runs.

The model does not call the tool. It **requests** the call.

THE MODEL

Decides

All it does is write one more piece of text:

```
read_file("sales.csv")
```

THE PROGRAM RUNNING THE LOOP

Does

It reads that request and runs the **actual code** behind the tool.

written by developers — so opening a file always means the same thing

AI picks the move, code does the work — **best of both worlds.**

The Loop, Powered By Text

The agent loop is **that one capability**, used over and over.

... and it thinks again. **That is the agent loop.**

Text generation, pointed at a goal — from one input you handed it.

What An LLM Can’t Do Alone

Brilliant at text. On its own, that is **all** it does.

It only writes text

It can’t open your file, send the mail or run anything.

It forgets everything

Every call starts from scratch — the whole chat gets re-sent each time.

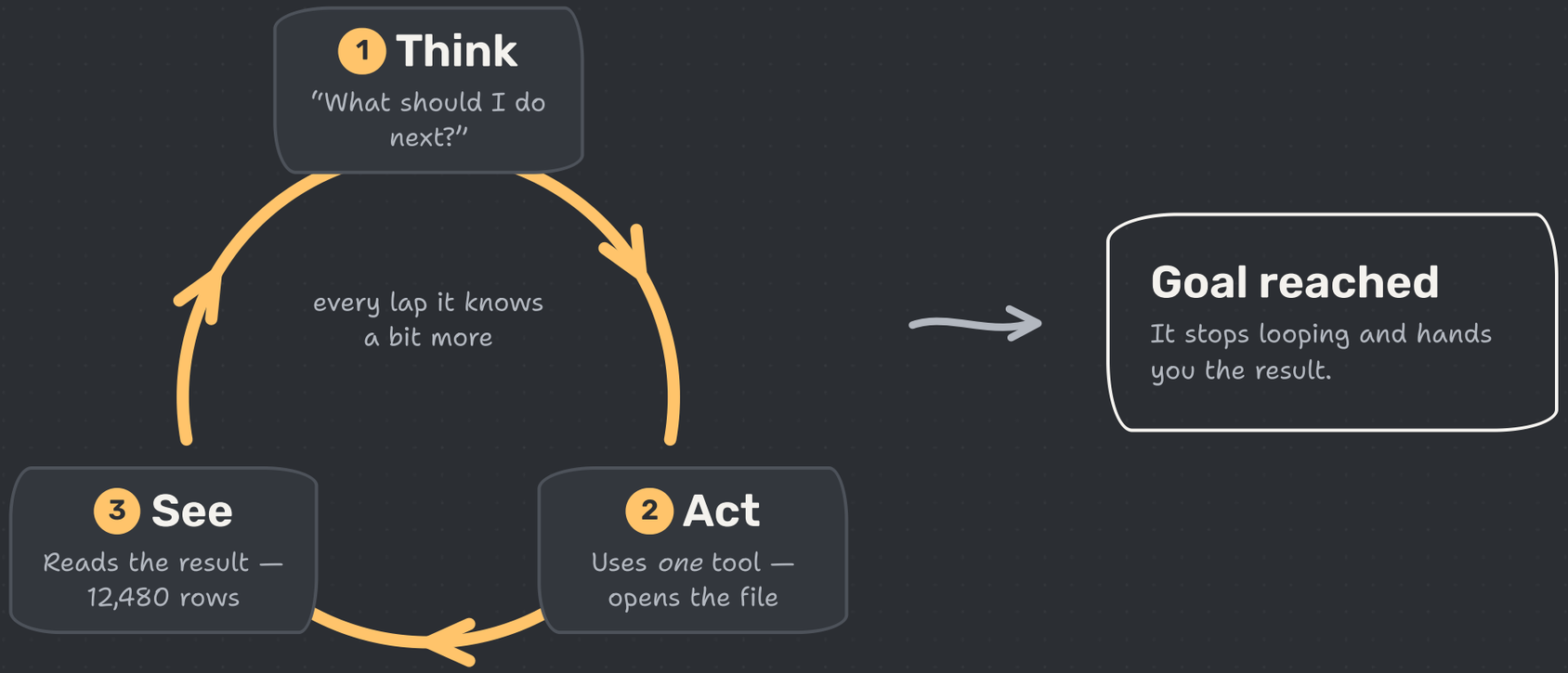
Its knowledge is frozen

It doesn’t know today, your files or your systems.

Hand it tools and wrap it in a loop, and all three go away. **That’s an agent.**

What’s an Agent Loop?

It works the way you do on a big task: one small step at a time, checking after each one.



Building Blocks & Context Engineering

The agentic loop — the engine that keeps everything below running.

everything else is what you hand it

A model

The brain doing the deciding

Instructions

Its job description & rules

Tools

What it can actually *do*

Skills

Know-how it opens when needed

Memory

What it carries between steps

A sandbox

A safe place to run & break things

Nearly all of it becomes context: the text the model reads before it decides. Choosing what goes in is context engineering.

Humans in the Loop

Some actions should not happen until **a person has agreed to them**.

The model asks for a tool → The harness pauses → A person approves or refuses

Built into the agent

Designing a task-specific agent means deciding which actions are too consequential to take on their own.

- issue a refund
- send the payment
- delete the record

the gate is part of the product

Offered by the tool

General agents ask before they touch anything. Most let you approve once, approve always, or turn the prompts off entirely.

- write a file
- run a command
- reach the network

configurable — and often switched off

Every prompt you auto-approve trades **oversight for speed**.

A refusal is not an error: it goes back as the tool's result, and the model plans around it.

That Program Is The Harness

An agent is **two pieces of software**. Only one of them runs on your side.

RUNS WHERE YOU ARE

The Harness

- builds the text sent to the model
- runs the tool the model asks for
- keeps the whole history
- decides whether to loop again

every app you saw at the start is one

your own code

RUNS ON A SERVER

The Model

It never touches your files, your inbox or the web.

IN	Everything known so far, as text
OUT	Text — an answer, or a request to call a tool

reached over an API

One lap of the loop is one request out, one response back.

What The Harness Adds

You type one line. The model receives **thousands of tokens**.

system instructions	Who the agent is, how it should behave, what it must not do. Written by whoever built the harness.	≈2,000
tool definitions	One block per tool: its name, a description of what it does and when to use it, and every parameter it takes. Usually the largest part by far. `read_file` `write_file` `run_code` `web_search` `list_directory` … and a dozen more	≈5,000
project instructions	Your AGENTS.md, any skill it has opened, anything it remembered.	≈800
your prompt	"Give me summary statistics for sales.csv"	12

And all of it is re-sent **every lap** — not just the first one.

Token counts are approximate and vary by harness. Your prompt is drawn taller than its true share so it stays readable.

How A Tool Is Described

A tool reaches the model as text: a name, a description, and its parameters.

WHAT THE MODEL IS GIVEN

name	`read_file`
description	Reads a file from the local filesystem and returns its contents. Use it before editing a file, or whenever the user refers to a file by name.
parameters	`path` `string, required` — absolute path to the file `limit` `number, optional` — maximum lines to read

WHAT IT IS NEVER GIVEN

```
read_file(path, limit)

  open the file
  read up to `limit` lines
  return the text
```

The harness holds the code and runs it. The model only ever reads the description beside it.

Reword that description, and the model uses the tool **differently**.

Getting An Agent

The loop is the same in all four cases. What changes is **how much of it you build**.

Talk to one

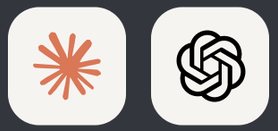
Somebody else built it, hosts it and pays for it. You send input.

- support agent on a site
- an agent a colleague built

someone else owns the loop

Run a ready-made tool

Install it, point it at your work, steer it with a prompt.



the tool owns the loop

Assemble one, no code

Drag the steps together, with an agent node in the middle.

- n8n
- Zapier Agents
- Make

the service owns the loop

Write it yourself

Call the model, run the tool it asks for, hand back the result, repeat. With a framework or without.

- Python
- TypeScript
- LangGraph

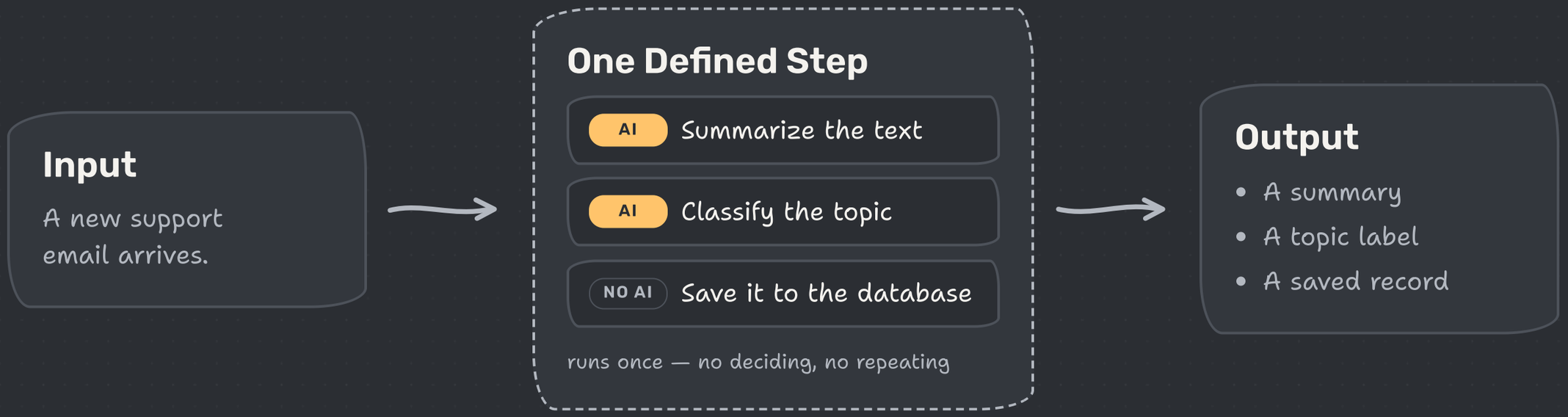
you own the loop

nothing to build — full control

Agents vs Workflows

A workflow is a deterministic flow of clearly defined actions.

You decide every step up front — the same input always takes the same path.



Workflow — you choose the steps. AI may power a step, but never picks the path.

Agent — the LLM chooses the steps, and how many of them it takes.

So an AI agent is really just a workflow whose middle step is an agentic loop.

Steering & Controlling AI Agents

Making the agent do what it *should* do

Three Ways To Steer It

A capable agent will always do *something*. Steering is how you make it do **the right thing**.

You never edit the loop — you change what it knows, and what it can reach.

Instructions

A file of standing rules, read at the start of every single session.

- AGENTS.md
- CLAUDE.md

Always in context

Skills

Folders of know-how the agent opens only when the task actually calls for it.

- SKILL.md
- scripts
- references

Loaded on demand

Tools & environment

What it may call, and what it can see. The only *hard* limit of the three.

- permissions
- sandbox
- tool set

Enforced, not requested

AGENTS.md — A README For Agents

One predictable place for the context an agent needs — so your README stays for humans.

```
my-project/
├── README.md          ← for people
├── AGENTS.md          ← for agents
└── packages/api/
    └── AGENTS.md      ← wins here
```

The closest file to the edited file wins. Your chat prompt overrides all of them.

- Plain Markdown. **No required fields**, no schema — any headings you like.
- Build and test commands, project layout, conventions — what you would otherwise retype every session.
- Nested files for monorepos, so each package can speak for itself.
- Adopted by **60k+ open-source projects** and 30+ agent tools.

It is context, not configuration — guidance the model reads, not a rule the software enforces.

They Stack — Closest Wins

Instructions come from several places at once, and they are **concatenated, not overridden**.

`org policy`	Company standards, pushed by IT — the one layer you cannot switch off.
`~/.claude/`	You, everywhere. Personal preferences across every project on your machine.
`./AGENTS.md`	The project. Checked into git, so the whole team gets the same agent.
`./packages/api/`	This corner of the codebase. Read last, so it has the final word.

Read root first, working directory last — so the **closest file wins.**

AGENTS.md *or* CLAUDE.md?

Same idea, two filenames. **Each tool reads its own** — and only its own.

> **AGENTS.md** — the shared convention. Codex, Copilot, Cursor, Gemini CLI, Jules and dozens more look for this name.

> **CLAUDE.md** — what Claude Code looks for. It does *not* read AGENTS.md.

CLAUDE.md

```
@AGENTS.md

## Claude Code
Use plan mode for changes under src/billing/.
```

One import bridges the two — every tool reads the same rules, written once. A symlink works too.

Write the instructions once. **Never maintain two copies** that drift apart.

What Belongs In It

It is loaded into **every session** — so every line you add is paid for on every task.

Put it here

Facts that are true for every task in this project.

- Build, test and lint commands
- Where things live — "API handlers are in src/api/"
- Conventions that differ from the tool defaults
- Mistakes you have had to correct twice

Aim for under ~200 lines

Put it elsewhere

Anything that only matters some of the time.

- A multi-step procedure → make it a **skill**
- Rules for one file type or folder → scope them to that path
- "Always run X before Y" → a **hook**, so it actually runs
- Things the agent can just read off the codebase

Longer files, weaker adherence

A Skill Is Just A Folder

Procedural know-how, packaged so it can be **version-controlled and shared** like any other file.

```
pdf-processing/
├── SKILL.md      ← required
├── scripts/      ← code it can run
├── references/   ← docs it can open
└── assets/       ← templates
```

```
---
name: pdf-processing
description: Extract PDF text, fill
  forms, merge files. Use when
  handling PDFs.
---

## Steps
1. …
```

name and **description** are the only required fields

license · compatibility · metadata · allowed-tools

The description is not documentation — it is **how the agent decides to reach for it.**

Loaded Only When Needed

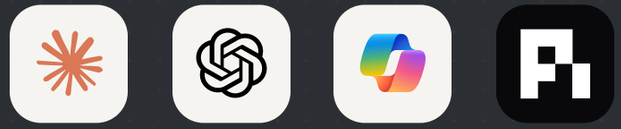
A hundred skills on hand, and almost none of them in the way. This is progressive disclosure.

1	**Discovery** — at startup, only each skill's name and description. Just enough to know it exists.	~100 tokens each
2	**Activation** — the task matches a description, so the agent opens the full SKILL.md.	< 5,000 tokens
3	**Execution** — it follows the instructions, opening references or running scripts only where they are needed.	as needed

The same trick the loop itself uses: pull context in late.

Write Once, Run Anywhere

Skills started at Anthropic and were released as an **open standard** — a folder and a Markdown file, nothing proprietary.



Cursor · Gemini CLI · VS Code · Junie · Goose · OpenCode · Kiro · & ~40 more

The skill you write today **outlives the tool** you wrote it in.

Instructions *vs* Skills

Both are Markdown you write. The difference is **when the agent reads them**.

Instructions — always loaded, whether or not this task needs them. Best for the handful of facts that are true every time.

Skills — loaded when the task matches. Best for the long procedure you only run on Fridays.

If it doesn't apply to every task, it doesn't belong in the file that loads on every task.

Configured General *vs* Task-Specific

Give a general agent the right instructions, skills and tools and it does the job. So **what is left of the difference?**

General, well configured		Task-specific
Broad — a shell, a file system, the web	TOOLS	Narrow — the few calls this job needs
Your machine, your repo, your accounts	ENVIRONMENT	Restricted — only what the job touches
Files you edit, and can change tomorrow	INSTRUCTIONS	Written once, into the product
You do — per task, in the prompt	WHO STEERS	Whoever built it, before you arrived
Does the wrong thing — you notice and correct	WHEN IT FAILS	**Can't do the wrong thing — can't adapt either**

Asking *vs* Enforcing

The honest limit of everything on the last few slides.

Instructions & skills ask

They are text in the context window. The model reads them and *usually* complies — more reliably when they are short and specific, less so when two of them disagree.

Good enough for conventions

Tools & environment enforce

A tool the agent was never given is a tool it cannot call, whatever it decides. A file outside the sandbox cannot be read, however hard it tries.

The only real guarantee

A task-specific agent isn't smarter. It is the same loop in a smaller world.

Understanding & Building AI-powered Workflows

Diving into an example to understand the WHAT and HOW

What You'll Learn

Everything we build here runs **on your own machine**, for you

You run it locally

You'll install a couple of tools and start it from your own terminal. Nothing here needs a server, an account, or a deployment.

A locked-down work laptop may not let you install — a personal machine is the easier place to follow along

It'll stay local

In practice, workflows usually run on a remote server. Getting them there — deployment, server management, the accounts and keys around it — is a separate skill.

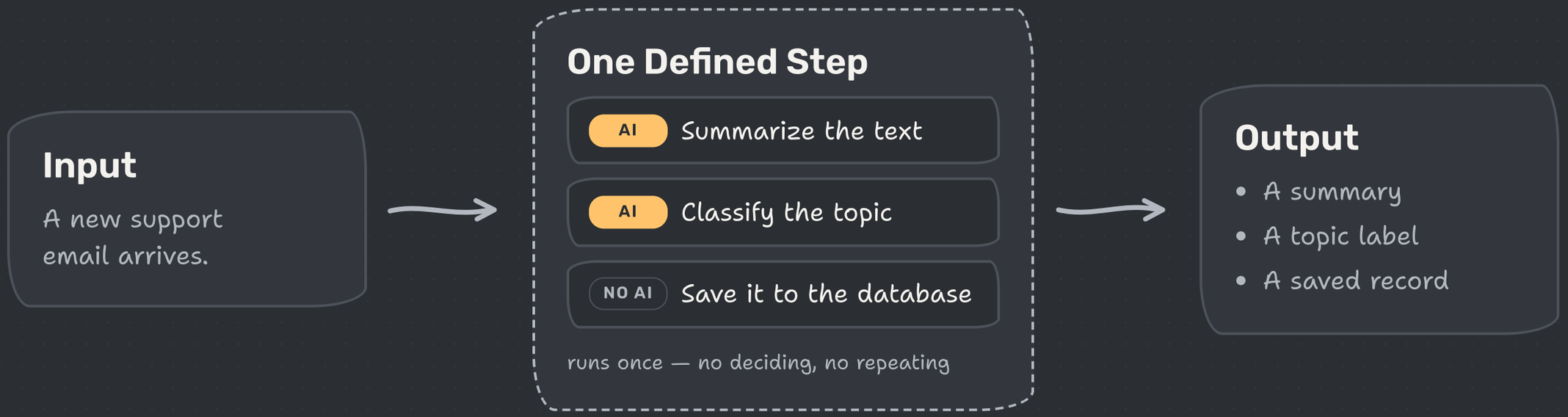
A real skill set, just a different one — not part of this course

We stay where the learning is: **making it work, and seeing why.**

What Are AI Workflows?

A workflow is a deterministic flow of clearly defined actions.

You decide every step up front — the same input always takes the same path.



AI Workflows *vs* AI Agents

Both may use AI (Agents definitely do).
The difference is **how much the LLM is allowed to decide.**

AI workflow		AI agent
You do — once, when you build it	WHO DECIDES	The model does — on every lap
Fixed, and known before it runs	THE STEPS	Unknown until it stops
None. Your code calls them, in an order you wrote	TOOLS	Exposed to the model, which asks for them by name
Read text, write text — summarize, classify, extract	THE MODEL'S JOB	Choose what happens next
Same path, every time — it is deterministic	RUN IT TWICE	It may take a different route to the same goal

So an AI agent is a workflow whose middle step is **an agentic loop.**

Understanding & Building AI Agents

Diving into an example to understand the WHAT and HOW

What You'll Learn

Same deal as the workflows: the agent we build runs **on your machine**, started by you.

The loop is real and the tools really run — it just isn't a product yet.

You run it locally

A file you start from your terminal, with your own API key. It reads your files and calls your tools, and stops when you close it.

You'll need to be able to install developer tools — a personal machine is the easier place to follow along

It'll stay local

In practice, task-specific agents usually run on a remote server. Getting one there — deployment, server management, the keys, permissions and running costs — is a separate skill.

A real skill set, just a different one — not part of this course

Understand the loop first. **Everything after that is plumbing.**

What We'll Look At

We'll look behind the scenes of a task-specific AI agent.

Why this one

A narrow agent strips the picture back to the parts that actually make it an agent — the loop, the tools, the decision. Nothing else in the way.

Fewer moving parts to follow

And the general ones?

Exactly the same machine. Just more tools on the model's list, and a bigger harness around the outside.

same loop · more tools · bigger harness

Nothing new to learn later

Learn it small here, and the big ones stop being mysterious.

Building AI Agents & Workflows

From consumer to producer

Options For Building

Three routes to the same result.

Visually, no code

You assemble the agent on a canvas, connecting one step to the next. The result is a diagram that documents itself.

n8n · Make · Flowise · Copilot Studio

maintainable without a developer

With code

You implement it in a programming language, with as much or as little library support as you choose.

Python · TypeScript · SDKs · frameworks

maximum control, maximum maintenance

Vibe coding

You describe the intended behaviour in natural language and a model produces the implementation.

Lovable · Replit Agent · Relevance Invent · Gumloop

the fastest route to a working prototype

Who Builds It, Where It Runs

Two questions place every option: **who builds it**, and **where the loop runs**.

	RUNS ON A USER'S MACHINE	RUNS ON A REMOTE SERVER
	Started by the person at the keyboard Reaches their files, their logins, their machine One session at a time Tends to suit general agents	Started by a message, an event or a schedule Reaches your systems, your data, your APIs Many sessions in parallel Tends to suit task-specific agents
YOU BUILD IT	**Built by you, run locally** You write or assemble the agent and execute it on your own hardware. a script · a command-line tool · a local n8n instance	**Built by you, deployed** The same agent, placed on infrastructure that you or a provider operates. n8n Cloud · Claude Managed Agents · Bedrock AgentCore
THIRD PARTY BUILDS IT	**A product you install** A vendor builds and maintains it; you run it against your own files and accounts. Claude Cowork · Claude Code · Copilot in your editor	**A hosted agent you subscribe to** A vendor builds, hosts and operates it. You configure it and connect your systems. Sierra · Agentforce · ChatGPT on the web

A tendency, not a constraint: a general agent may equally run on a shared server.

Building It Visually

The flow is assembled rather than written. Each node is one step, and the connections carry data between them.

THE CANVAS IS THE WORKFLOW — ONE NODE IS THE AGENT

That single node contains the same loop described earlier. You select the model and enable the tools; the platform generates their descriptions.

Automation platforms

n8n · Make · Zapier · Activepieces

an agent node within a larger workflow

Agent canvases

Flowise · Langflow · Dify

the agent is the primary object

Inside a suite

Copilot Studio · Agentforce · Vertex AI Agent Builder

integrated with an existing platform

Options When Building With Code

Three levels, each delegating more of the machinery.

		THE LOOP	THE STATE
From scratch	You call the model's API directly and implement the loop yourself.	you	you
SDKs & libraries	These handle transport, tool definitions and streaming. Several will also run the loop on your behalf.	shared	you
Frameworks	The framework owns the loop and holds the conversation, the progress and the retries, invoking your code as needed.	framework	framework

The distinction between a library and a framework is not well defined. **The useful question is which one owns the loop**:

Anthropic & OpenAI SDKs · Vercel AI SDK · LangGraph · CrewAI · Mastra · eve

Provider-Side Or Your Side

The same question applies to **each individual tool**.

Defined and executed by you

You author the description and implement the code. The model requests the call; your harness executes it.

`read_file` `send_invoice`

you define it, you run it

Defined by them, executed by you

The provider defines it and the model is trained on it. Your harness still executes every call.

`bash` `text_editor` `computer`

they define it, you run it

Defined and executed by them

The provider executes it inside its own loop. The call never reaches your harness; only the result does.

`web_search` `code_execution`

executed remotely; the run may pause mid-task

THE CONVERSATION HISTORY: PRUNE IT YOURSELF, OR HAVE THE PROVIDER COMPACT IT

WHAT MANAGING IT YOURSELF REQUIRES

- ! **Thinking blocks are returned empty.** They must be sent back exactly as received; discarding one because it appears blank causes the next call to be rejected.
- ! **Tool calls must remain paired.** Truncating the history between a request and its result also causes a rejection.
- ! **The provider's summary is carried in the response.** Retaining only the text discards what it needs to reconstruct the history.

Which Building Blocks Will You Support?

The agentic loop — the engine that keeps everything below running.

everything else is a decision, and each one adds scope

A model

The brain doing the deciding

A provider to choose, and a bill that grows with every lap.

Instructions

Its job description & rules

The least expensive item here, and the usual starting point.

Tools

What it can actually *do*

One integration, and its failure modes, for every tool.

Skills

Know-how it opens when needed

The procedures must be written, and kept accurate.

Memory

What it carries between steps

Storage, and a policy for what is worth retaining.

A sandbox

A safe place to run & break things

Isolation that you must provide and fund.

What The Platforms Sell You

The feature list of any agent platform is **a list of the difficult parts**.

Approvals
Pausing for human confirmation before an irreversible action.

Observability
Inspecting what the loop did, run by run.

Evaluations
Establishing whether a change improved or degraded behaviour.

Durability
Surviving a crash, a restart or a deployment mid-run.

Sessions
Resuming a conversation hours or days later.

Credentials
Keys the agent can use but never read.

Cost & limits
A ceiling, before a runaway loop exhausts the budget.

Injection defence
Content encountered mid-task that attempts to issue instructions.

Isolation
One customer's run never reaching another's.

Each one is either a feature you buy or **a project you run**.

Building Agents with Libraries & Frameworks

You *can* do it all on your own — but you don't have to

Some Options Worth Knowing

Every part you would otherwise build is already written by someone.

Vercel AI SDK

One API across every provider, streaming and tool calls included.

TypeScript

Mastra

A whole agent backend: memory, workflows, approvals, evals.

TypeScript

LangGraph

The run as an explicit graph, with checkpoints you can resume.

Python & TypeScript

OpenAI Agents SDK

Small, clean primitives: tools, handoffs, guardrails, tracing.

Python & TypeScript

PydanticAI

Typed agents with validated output, in a normal Python service.

Python

CrewAI

Agents as a team of roles: researcher, writer, reviewer.

Python

LlamaIndex

For agents whose real problem is documents and private data.

Python

Google ADK

Multi-agent applications for the Gemini and Google Cloud world.

Python

Claude Agent SDK

Claude Code's own harness as a library: files, shell, sandbox.

Python & TypeScript

And plenty more, each with its own centre of gravity:

Microsoft Agent Framework · Agno · smolagents · Strands Agents · eve